

Error Log

Noorinder

Credit Name: Chapter 13

Assignment Name: QueueList

1. Incorrect Front Assignment During Enqueue

- **Error:** When the queue was empty, the **front** pointer was not assigned to the newly added node, leaving the queue in an invalid state.
- **Cause:** The **enqueue()** method did not properly handle the case where both **front** and **rear** should point to the same node for an empty queue.
- **Fix:** Modified the **enqueue()** method to assign the new node to both **front** and **rear** when the queue is empty.

Before Fix:

```
public void enqueue(Object item) {  
    Node newNode = new Node(item);  
    rear.setNext(newNode); // rear could be null, causing a NullPointerException  
    rear = newNode;  
}
```

After Fix:

```
public void enqueue(Object item) {  
    Node newNode = new Node(item);  
    if (isEmpty()) { // When the queue is empty  
        front = newNode; // Set front to the new node  
        rear = newNode; // Also set rear to the new node  
    } else {  
        rear.setNext(newNode); // Link the new node to the existing rear  
        rear = newNode; // Update rear to the new node  
    }  
}
```

2. NullPointerException on Dequeue from Empty Queue

- **Error:** The `dequeue()` method did not check if the queue was empty before attempting to remove an element, leading to a `NullPointerException`.
- **Cause:** The method assumed that the `front` node always existed, even when the queue was empty.
- **Fix:** Added a check to ensure the queue is not empty before attempting to dequeue an element.

Before Fix:

```
public Object dequeue() {
    Object data = front.getData(); // Causes NullPointerException if queue is empty
    front = front.getNext();
    return data;
}
```

After Fix:

```
public Object dequeue() {
    if (isEmpty()) { // Check if the queue is empty
        System.out.println("Queue is empty. Cannot dequeue."); // Return an error message
        return null;
    }
    Object data = front.getData(); // Get the front item
    front = front.getNext(); // Move front to the next node
    if (front == null) { // If the queue is now empty
        rear = null; // Update rear to null as well
    }
    return data; // Return the dequeued item
}
```

3. Unlinked Nodes During Enqueue

- **Error:** New elements added to the queue were not properly linked to the previous `rear` node, causing the queue's continuity to break.
- **Cause:** The `rear` node's `next` reference was not updated to point to the new node.
- **Fix:** Updated the `enqueue()` method to set the `rear` node's `next` reference to the new node before updating the `rear` pointer.

Before Fix:

```

public void enqueue(Object item) {
    Node newNode = new Node(item);
    rear = newNode; // Does not link newNode to the existing queue
}

```

After Fix:

```

public void enqueue(Object item) {
    Node newNode = new Node(item);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.setNext(newNode); // Link the new node to the current rear
        rear = newNode;        // Update the rear reference
    }
}

```

4. Size Calculation Inefficiency

- **Error:** The `size()` method traversed the entire queue every time it was called, leading to inefficient performance for large queues.
- **Cause:** The queue size was calculated dynamically by iterating through all nodes instead of maintaining a running count.
- **Fix:** Added a `size` variable that increments or decrements during `enqueue()` and `dequeue()` operations for constant-time size retrieval.

Before Fix:

```

public int size() {
    int count = 0;
    Node current = front;
    while (current != null) {
        count++;
        current = current.getNext();
    }
    return count;
}

```

After Fix:

```

private int size; // Add a size variable to track the number of elements

public QueueList() {
    front = null;
    rear = null;
    size = 0; // Initialize size to 0
}

public void enqueue(Object item) {
    Node newNode = new Node(item);
    if (isEmpty()) {
        front = newNode;
        rear = newNode;
    } else {
        rear.setNext(newNode);
        rear = newNode;
    }
    size++; // Increment size when adding an element
}

public Object dequeue() {
    if (isEmpty()) {
        System.out.println("Queue is empty. Cannot dequeue.");
        return null;
    }
    Object data = front.getData();
    front = front.getNext();
    if (front == null) {
        rear = null;
    }
    size--; // Decrement size when removing an element
    return data;
}

public int size() {
    return size; // Return the size variable directly
}
}

```

5. Improper Error Handling in Peek

- **Error:** The `peek()` method attempted to access the front node even when the queue was empty, causing runtime exceptions.
- **Cause:** The method did not check whether the queue was empty before accessing the `front` node.
- **Fix:** Added a check to ensure the queue is not empty before returning the front element.

Before Fix:

```
public Object peek() {  
    return front.getData(); // Causes NullPointerException if queue is empty  
}
```

After Fix:

```
public Object peek() {  
    if (isEmpty()) { // Check if the queue is empty  
        System.out.println("Queue is empty. No front item to peek."); // Error message  
        return null;  
    }  
    return front.getData(); // Return the data at the front  
}
```
